SIMATS ENGINEERING
ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I **K.Sasidhar Reddy(19252540)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *K.Sasidhar Reddy*

Assessment Tasks
Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.
   Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
   Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.

Explain how identity management and encryption should be implemented.
Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.

Cloud Computing – Case Study Answers

5 Questions × 5 Marks = 25 Marks

Question 1: Cloud Migration of a Video Streaming Platform

> **Scenario:** *A media company wants to migrate its video streaming platform to the cloud, handling millions of concurrent users with minimal buffering.*

1. Proposed Architecture

The platform is split into an ingest/processing pipeline, a storage layer, a delivery layer and a stateless application layer:

- **Ingest and transcoding:** Uploaded masters land in object storage (e.g., Amazon S3 / Azure Blob / Google Cloud Storage). A queue-driven transcoding fleet (AWS MediaConvert, Elastic Transcoder or containerised FFmpeg workers) creates multiple bitrate renditions (240p–4K) and packages them as HLS/DASH segments for Adaptive Bitrate Streaming (ABR).
- **Storage:** Segments and manifests are kept in durable object storage with lifecycle policies; metadata (catalogue, users, watch history) lives in a managed NoSQL/relational database such as DynamoDB, Cassandra or Aurora with read replicas.
- **Delivery:** A global Content Delivery Network (CloudFront, Akamai, Cloudflare) caches segments at edge locations close to viewers, with an origin shield protecting the storage origin.
- **Application layer:** Stateless microservices (authentication, recommendations, playback, billing) run on Kubernetes or serverless functions behind API gateways.
- **Security:** Signed URLs/cookies, DRM (Widevine, FairPlay, PlayReady) and TLS to protect content.

2. Scalability and Caching Techniques

- **Horizontal scaling:** All services are stateless so more instances can be added instantly; sessions are held in Redis/ElastiCache rather than on servers.
- **Multi-tier caching:** Browser/device cache → CDN edge cache → regional/mid-tier cache → origin shield → object storage. Popular titles reach a cache-hit ratio above 95%, so only a small fraction of traffic ever reaches the origin.
- **In-memory caching:** Redis/Memcached caches metadata, recommendations and user sessions to cut database load.
- **Adaptive bitrate and short segments:** 2–6 second segments let the player switch quality instantly with bandwidth changes, which minimises buffering.
- **Pre-positioning:** New releases and trending content are pushed to edge locations before peak hours (cache warming).

3. Role of Load Balancing and Auto-Scaling

- **Global load balancing:** DNS-based routing (Route 53, Azure Traffic Manager) with latency or geo policies directs each viewer to the nearest healthy region/CDN point of presence.

- **Layer 7 load balancing:** Application Load Balancers distribute API and playback-session requests across instances, using health checks to remove failing nodes and path-based routing to separate services.
- **Auto-scaling:** Policies based on CPU, requests per second, concurrent streams or queue length add instances (Kubernetes HPA / cloud auto-scaling groups) as load rises and remove them as it falls. Scheduled scaling prepares for known events such as a live sports match or a series premiere, while predictive scaling handles daily patterns.

Together, load balancers spread traffic evenly and auto-scaling keeps capacity matched to demand, so no server becomes a bottleneck, startup time stays low and buffering is minimal even during sudden spikes.

4. Cost Optimization Strategies

- **Right-sizing and instance mix:** Reserved Instances / Savings Plans for the steady baseline, spot instances for fault-tolerant transcoding jobs (up to ~90% cheaper), and on-demand only for bursts.
- **Storage tiering:** Move older, rarely watched titles to Infrequent Access/Glacier tiers using lifecycle rules; store only the renditions actually requested.
- **CDN cost control:** Maximise cache-hit ratio, negotiate committed-use CDN pricing, use multi-CDN to route to the cheapest performing provider, and compress manifests.
- **Efficient encoding:** Per-title/content-aware encoding and modern codecs (H.265/AV1) reduce bitrate, and hence bandwidth cost, at the same quality.
- **Data-transfer optimisation:** Keep traffic within a region, use origin shield to reduce egress and avoid unnecessary cross-region replication.
- **FinOps practices:** Tagging, budgets, cost dashboards and anomaly alerts; scale down non-production environments out of hours.

Question 2: Multi-Cloud Container Deployment for a Fintech Startup

> **Scenario:** *A fintech startup deploys containerised applications across multiple cloud providers and struggles with consistency and deployment management.*

1. Proposed Solution: Containers + Orchestration + Multi-Cloud

- **Standardise on containers:** Package every service as an immutable OCI image stored in a central registry (Harbor, or replicated ECR/ACR/GCR) so the same artifact runs everywhere.
- **Kubernetes as the common layer:** Use managed Kubernetes (EKS, AKS, GKE) on each provider, giving a uniform API regardless of cloud. A multi-cluster management tool (Rancher, Anthos, Azure Arc, Red Hat ACM) provides one control plane for all clusters.
- **Infrastructure as Code:** Terraform/Crossplane for provisioning identical networks, clusters and policies on every cloud.
- **GitOps delivery:** Argo CD or Flux keeps cluster state in sync with a Git repository, so every cluster is deployed from one source of truth, with full audit trail and easy rollback.
- **CI/CD pipeline:** Build → scan → sign → push → deploy, with Helm/Kustomize overlays for environment differences.
- **Policy and consistency:** Open Policy Agent/Gatekeeper and Kyverno enforce the same security and compliance rules (important for fintech regulations such as PCI-DSS) on all clusters.
- **Observability and secrets:** Prometheus/Grafana/OpenTelemetry for unified monitoring; HashiCorp Vault for centralised secret management.

2. Service Discovery and Scaling

- **Within a cluster:** Kubernetes Services and CoreDNS give each service a stable DNS name and virtual IP; kube-proxy/Endpoints automatically track healthy pods.
- **Across clusters and clouds:** A service mesh (Istio, Linkerd) or Consul with multi-cluster federation provides cross-cluster discovery, mutual TLS, retries, circuit breaking and traffic splitting. Global DNS/GSLB (e.g., Cloudflare, Route 53) steers external users to the closest or healthiest cluster.
- **Pod scaling:** Horizontal Pod Autoscaler scales replicas on CPU, memory or custom metrics; KEDA scales on events such as queue depth.
- **Node scaling:** Cluster Autoscaler or Karpenter adds/removes worker nodes per cluster as pods become unschedulable or nodes sit idle.
- **Right sizing:** Vertical Pod Autoscaler recommends resource requests/limits; Pod Disruption Budgets keep services available during scale-down.

3. Benefits and Risks of Multi-Cloud

Benefits	Risks
Avoids vendor lock-in and improves bargaining power	Higher operational complexity and need for broader skills

Benefits	Risks
Higher resilience: survives an outage of one provider	Inconsistent services, APIs and tooling between providers
Best-of-breed services and regional coverage	Larger attack surface and harder security/identity governance
Helps meet data-residency and regulatory demands	Data egress charges and latency for cross-cloud traffic
Cost optimisation by workload placement	Hard to maintain cost visibility and avoid duplicated spend

Question 3: Backup and Disaster Recovery for an E-Learning Platform

> **Scenario:** *An e-learning platform suffers data loss due to accidental deletion in the cloud and needs a robust backup and disaster recovery (DR) strategy.*

1. Cloud-Native Backup, Replication and Versioning Design

- **Versioning and soft delete:** Enable object versioning on storage buckets (S3 Versioning, Azure soft delete) so any deleted or overwritten file can be restored. Add MFA-Delete / object lock (WORM) for critical content such as course material and student records.
- **Database protection:** Automated snapshots plus continuous point-in-time recovery (PITR) for managed databases (RDS/Aurora, Cloud SQL); deletion protection turned on for production databases.
- **Centralised backup service:** AWS Backup / Azure Backup / Google Backup and DR schedules and retains backups of volumes, databases and file systems under one policy, following the 3-2-1 rule (3 copies, 2 media types, 1 off-site).
- **Replication:** Cross-region replication (CRR) of buckets and read replicas of databases in a second region; backups copied to a separate account (isolated vault) so a compromised or mistaken admin cannot delete everything.
- **Access control:** Least-privilege IAM, separate roles for delete/restore, approval workflows and audit logging (CloudTrail) to prevent and trace accidental deletions.
- **Infrastructure as Code:** Environment defined in Terraform/CloudFormation so it can be rebuilt quickly in another region.
- **Lifecycle management:** Retain frequent recent backups, move older ones to cheaper archive tiers and expire them per policy.

2. How RTO and RPO Are Achieved

RPO (Recovery Point Objective) is the maximum acceptable data loss measured in time; **RTO (Recovery Time Objective)** is the maximum acceptable downtime.

Objective	Technique	Example Outcome
RPO	Continuous replication, PITR, frequent snapshots, versioning	Near-zero to a few minutes for databases; last version for files
RTO	Warm standby/pilot-light region, automated runbooks, IaC, pre-built images, DNS failover	Minutes to a few hours depending on tier

Tiering applications by criticality balances cost: the login and course-delivery services get a warm standby (RTO of minutes, RPO of seconds), whereas archived reports use backup-and-restore (RTO of hours, RPO of 24 hours).

3. Steps to Ensure Business Continuity During Failures

1. **Detect:** monitoring and alerts (CloudWatch/Azure Monitor) identify the failure or deletion immediately.
2. **Declare and communicate:** follow the incident plan, notify the response team, and post status updates to students and instructors.

3. **Contain:** stop further deletions, revoke compromised credentials and freeze affected resources.
4. **Restore:** recover files from previous versions, roll databases back using PITR, or fail over to the standby region.
5. **Redirect traffic:** update DNS/load balancer routing so learners reach the healthy environment; use a cached/static “read-only” mode for videos and notes if needed.
6. **Validate:** run integrity and smoke tests on the restored data and applications.
7. **Review and improve:** perform a post-incident analysis, update runbooks and test DR drills regularly (at least twice a year).

Question 4: Hybrid Cloud Architecture for a Government Agency

> **Scenario:** *A government agency adopts a hybrid cloud: sensitive data and some compliance-bound applications stay on-premises, while non-sensitive workloads move to the public cloud.*

1. Hybrid Architecture with Secure Communication

- **Workload placement:** Classified/personal citizen data, legacy and regulated systems remain in the on-premises private cloud/data centre; public-facing portals, analytics on anonymised data, development/test, backups and burst capacity run in a (preferably government-accredited) public cloud region.
- **Secure connectivity:** Dedicated private links (AWS Direct Connect, Azure ExpressRoute, Google Interconnect) backed up by redundant IPsec site-to-site VPN tunnels; traffic never crosses the open internet for sensitive flows.
- **Network segmentation:** Separate VPCs/VNets and subnets, a hub-and-spoke topology, firewalls/next-gen firewalls, security groups, a DMZ and private endpoints for cloud services.
- **API and integration layer:** API gateways, message queues and a controlled data-transfer gateway ensure that only approved, filtered data moves between environments.
- **Consistent platform:** Kubernetes/VMware-based tooling (e.g., Azure Arc, AWS Outposts, Anthos) provides a uniform runtime across both sides.
- **Security monitoring:** A central SIEM (e.g., Splunk/Sentinel), IDS/IPS, DDoS protection and WAF with logs from both environments.

2. Identity Management and Encryption

Identity and Access Management

- Federated single sign-on: integrate on-premises Active Directory with the cloud identity provider (Azure AD/Entra ID, AWS IAM Identity Center) using SAML/OIDC and directory synchronisation.
- Mandatory multi-factor authentication, role-based and attribute-based access control, and least-privilege/just-in-time access.
- Privileged Access Management (PAM) for administrators, regular access reviews and complete audit logs.
- Zero-trust principle: verify every user, device and request regardless of network location.

Encryption

- **Data in transit:** TLS 1.2/1.3 for applications and IPsec/MACsec for links between sites.
- **Data at rest:** AES-256 encryption for storage, databases and backups.
- **Key management:** Keys held in an on-premises Hardware Security Module (HSM) or cloud KMS with customer-managed/“bring your own key”; regular key rotation and separation of duties so the cloud provider cannot read sensitive data.
- **Data protection:** Data classification, tokenisation or masking before any data is sent to the cloud, and DLP tools to block leaks.

3. Challenges in Managing Hybrid Cloud

- **Security and compliance:** Maintaining uniform policies and audit evidence across two environments; data sovereignty and legal requirements.
- **Integration complexity:** Incompatible legacy systems, different APIs and tools.
- **Network latency and bandwidth:** Delays and cost between on-premises and cloud components.
- **Visibility and monitoring:** Fragmented logs and metrics make troubleshooting and threat detection hard.
- **Skills and operations:** Teams must master both traditional and cloud technologies.
- **Cost management:** Duplicate infrastructure, data egress charges and unclear cost allocation.
- **Data consistency and governance:** Keeping data synchronised and classified correctly across locations.

Question 5: High Availability and Fault Tolerance for a SaaS Provider

> **Scenario:** *A SaaS provider must keep its application running even if an entire cloud region goes down.*

1. Multi-Region Deployment Strategy

- **Active-active (preferred for critical SaaS):** The application runs in two or more regions simultaneously; users are routed to the nearest healthy region, and each region can absorb the other's traffic.
- **Active-passive / warm standby:** A scaled-down copy runs in a secondary region and is scaled up on failure; cheaper but with a longer RTO.
- **Within each region:** Deploy across at least three Availability Zones with stateless application tiers behind load balancers and auto-scaling groups.
- **Global traffic management:** Global DNS with health checks and latency/geo routing (Route 53, Azure Front Door, Cloudflare) or anycast load balancing (Global Accelerator, Google Cloud Load Balancing).
- **Consistency of deployments:** Infrastructure as Code and CI/CD pipelines deploy identical stacks to every region; use canary/blue-green releases to avoid a bad release taking down all regions.
- **Loose coupling:** Queues, circuit breakers, retries with back-off and bulkheads prevent cascading failures.

2. Data Replication and Failover Configuration

- **Relational databases:** Cross-region read replicas or global databases (Aurora Global Database, Cloud Spanner, Azure SQL failover groups) with asynchronous replication (RPO of seconds) or synchronous replication where zero data loss is needed; replicas can be promoted to primary on failure.
- **NoSQL:** Multi-region, multi-master tables (DynamoDB Global Tables, Cosmos DB, Cassandra) with conflict-resolution rules (last-writer-wins or application-defined).
- **Object storage:** Cross-region replication of buckets, with versioning enabled.
- **Caches and sessions:** Store sessions in a replicated store or use stateless tokens (JWT) so users are not logged out on failover.
- **Automated failover:** Health checks (shallow and deep) trigger DNS/load-balancer failover with a short TTL (30–60 seconds); runbooks or automation promote the database replica and scale up the surviving region.
- **Failback:** After recovery, resynchronise data, test and then move traffic back gradually.
- **Testing:** Regular game days and chaos engineering (e.g., simulated region failure) validate RTO/RPO targets.

3. Monitoring and Alerting for Proactive Detection

- **Metrics:** Track the "golden signals": latency, traffic, errors and saturation, as well as CPU, memory, replication lag, queue depth and database connections (CloudWatch, Prometheus, Datadog).
- **Centralised logging and tracing:** ELK/OpenSearch, Cloud Logging and distributed tracing (OpenTelemetry, Jaeger) to locate faults across services and regions.

- **Synthetic monitoring:** Scripted probes from multiple locations simulate user journeys and catch regional problems before customers report them.
- **Real User Monitoring:** Measures actual page-load and API performance experienced by users.
- **Alerting:** Threshold and anomaly-based alerts tied to SLOs/error budgets; routed by severity to on-call engineers via PagerDuty/Opsgenie/Slack, with escalation policies and runbook links to reduce mean time to resolve.
- **Dashboards and status page:** Real-time dashboards for operations and a public status page for customer communication.
- **Continuous improvement:** Post-incident reviews, SLA reporting and capacity forecasting to prevent repeat issues.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented